KEVGraph: Proof-Carrying Rapid Remediation for Exploited Vulnerabilities in Open-Source Supply Chains

Daniel Okumu Omondi
Independent Researcher
Email: `(add email)`

***Abstract*—Open-source supply chain attacks increasingly exploit known vulnerabilities in transitive dependencies, yet defenders lack tools that produce *auditable*, *minimal*, and *independently verifiable* remediation plans optimized for reducing exploited exposure. Existing vulnerability scanners rank findings by severity but do not reason about reachability through dependency graphs, exploitation status, or time-to-risk-reduction under realistic change budgets. We introduce KEVGraph, a framework that constructs resolved npm dependency graphs from public data (lockfiles, deps.dev, OSV), joins them with CISA's Known Exploited Vulnerabilities (KEV) catalog as a ground-truth exploitation signal, and formulates remediation as a risk-time minimization problem over exploited reachable subgraphs. KEVGraph produces *remediation certificates*: minimal proof subgraphs that a third party can independently verify using the same public inputs to confirm that a prescribed set of upgrade actions breaks every exploited reachable path in the dependency model. We define the problem formally, present a greedy action-selection algorithm with bounded complexity, prove certificate soundness and risk-monotonicity, and describe a reproducible evaluation plan using public GitHub repositories, OSV, and KEV timeline data. All inputs are publicly available; no proprietary data is used.**

***Index Terms*—software supply chain security, vulnerability remediation, exploited vulnerabilities, dependency graphs, proof-carrying security, KEV, OSV, npm**

I. Introduction

On December 9, 2021, the disclosure of CVE-2021-44228 in Apache Log4j demonstrated that a single exploited vulnerability in a transitive dependency could propagate risk across thousands of downstream applications within hours [1]. The incident was not exceptional in kind—only in scale. The Cybersecurity and Infrastructure Security Agency (CISA) maintains the Known Exploited Vulnerabilities (KEV) catalog precisely because exploitation of known vulnerabilities in software supply chains is a persistent, systemic threat to U.S. critical infrastructure [2]. Executive Order 14028 on Improving the Nation's Cybersecurity explicitly calls for improved software supply chain security, software bills of materials (SBOMs), and timely remediation of known exploited vulnerabilities [3].

Despite this policy urgency, a practical gap persists between *vulnerability detection* and *auditable remediation*. Current tools—commercial scanners, `npm audit`, GitHub Dependabot—identify vulnerable packages and sometimes suggest upgrades. However, they typically:

1) Rank by CVSS severity rather than exploitation status, conflating theoretical risk with active threat.
2) Treat dependencies as flat lists rather than resolved transitive graphs, missing the distinction between reachable and unreachable exposure.
3) Produce remediation advice that is neither minimal nor independently verifiable—there is no artifact a third party (auditor, downstream consumer, regulator) can check to confirm that the prescribed actions actually eliminate the claimed exploited paths.
4) Do not optimize for *time-to-risk-reduction*, leaving defenders without guidance on which of many possible upgrades most rapidly reduces their exploited attack surface under limited change budgets.

This paper introduces KEVGraph, a framework that addresses these gaps for the npm ecosystem. The contributions are:

- **C1.** A formal *exploited-exposure* metric defined over resolved dependency graphs, grounded in KEV membership and graph reachability (Section V).
- **C2.** A *risk-time objective* that links KEV catalog update events to the defender's goal of minimizing the duration of exploited reachable exposure (Section V).
- **C3.** A *proof-carrying remediation certificate* format with an explicit verification procedure and minimality criteria, enabling independent third-party audit (Section VI).
- **C4.** A greedy action-selection algorithm for approximate risk-time minimization under change budgets, with complexity analysis and scaling strategies (Section VII).
- **C5.** Two formal properties—certificate soundness and risk monotonicity—with proof sketches (Section VIII).

All data inputs (npm lockfiles, deps.dev API, OSV API, CISA KEV catalog) are publicly available. No proprietary or employer-derived information is used.

II. Background

A. CISA Known Exploited Vulnerabilities (KEV) Catalog

The KEV catalog is a curated list maintained by CISA of CVEs for which there is reliable evidence of active exploitation [2]. Federal civilian agencies are required under Binding Operational Directive 22-01 to remediate KEV-listed vulnerabilities within specified timelines [4]. The catalog is published as a machine-readable feed, updated as new exploitation evidence is confirmed. Unlike CVSS scores, which estimate theoretical severity, KEV membership is a binary signal of *observed exploitation in the wild*.

B. Open Source Vulnerabilities (OSV) and deps.dev

The OSV database [5] aggregates vulnerability advisories from multiple sources into a normalized schema that maps CVE identifiers to affected package version ranges. The deps.dev service [6] provides resolved dependency graph data for packages across multiple ecosystems, including npm. Both services expose public APIs suitable for programmatic querying.

C. npm Dependency Resolution and Lockfiles

npm uses a deterministic dependency resolution algorithm that, given a `package.json` manifest, produces a `package-lock.json` lockfile specifying the exact version of every direct and transitive dependency [7]. The lockfile defines a rooted tree (with potential deduplication) that constitutes the resolved dependency graph for a project. This graph is the foundation for reachability analysis: a vulnerability in package p at version v is *reachable* from the project root if and only if there exists a path from the root to a node representing $p@v$ in the resolved graph.

D. Prior Work

Several lines of work relate to KEVGraph but do not address the specific combination of problems we target.

Vulnerability prioritization. Tools such as EPSS estimate exploitation probability from features of CVE records [8]. SSVC provides decision support for vulnerability response [9]. These systems rank vulnerabilities but do not produce remediation plans, do not reason about transitive reachability, and do not generate verifiable artifacts.

Dependency analysis. Eclipse Steady and related work perform reachability analysis for Java dependencies using call-graph construction [10]. Deps.dev provides dependency metadata [6]. These approaches do not combine exploitation ground truth with remediation optimization or certificate generation.

Supply chain security frameworks. SLSA defines provenance requirements for build artifacts [11]. NIST SP 800-218 (SSDF) and the NIST Cybersecurity Framework provide process-level guidance [12], [13]. These frameworks establish *what* organizations should do but not *how* to optimally sequence remediation or *how* to prove that remediation was effective.

SBOM formats. CycloneDX and SPDX define SBOM formats [14], [15]. SBOMs enumerate components but do not encode remediation reasoning or verification procedures.

To our knowledge, no published system combines KEV-grounded exploitation signals, transitive dependency reachability, time-optimal remediation planning, and independently verifiable proof artifacts into a single framework.

III. Threat Model and Scope

A. Adversary Model

We consider an adversary who exploits CVEs listed in the CISA KEV catalog against software systems with reachable transitive dependencies on the affected packages. The adversary's capability is bounded by the set of *known* exploited vulnerabilities; we do not model zero-day exploitation. The adversary's reach is determined by the dependency graph: exploitation succeeds only if a path exists from an application entrypoint to a node containing a KEV-listed vulnerability at an affected version.

B. Defender Model

The defender maintains one or more npm projects and seeks to minimize the duration of exploited reachable exposure. The defender faces a *change budget*: each upgrade action carries a cost (testing effort, compatibility risk, engineering time). The defender cannot apply all upgrades simultaneously and must prioritize.

C. Non-Goals

- **Zero-day vulnerabilities.** We restrict to KEV-listed CVEs, which have confirmed exploitation evidence.
- **Proprietary enterprise inventories.** All inputs are public.
- **Exploit construction or operational attack guidance.** This work is strictly defensive.
- **Call-graph-level reachability.** We model reachability at the package-dependency level, not the function level.

IV. Notation and Definitions

We introduce notation used throughout the paper.

Dependency graph. Let $G = (V, E)$ be a directed acyclic graph where V is a set of package-version nodes and $E \subseteq V \times V$ represents "depends on" relationships. Node $r \in V$ is the project root. For node $v \in V$, let $\mathrm{pkg}(v)$ and $\mathrm{ver}(v)$ denote its package name and version.

Entrypoints. Let $S \subseteq V$ be the set of entrypoint nodes (typically $S = \{r\}$ for a single-root project, but the formulation generalizes to workspaces).

Reachability. Node v is *reachable* if there exists a directed path from some $s \in S$ to v in G. Let $\mathrm{Reach}(G, S) \subseteq V$ denote the set of reachable nodes.

Vulnerability mapping. Let $\mathrm{vuln} : V \to 2^{\mathcal{C}}$ map each node to a (possibly empty) set of CVE identifiers affecting that package-version, obtained from OSV.

KEV labeling. Let $K \subseteq \mathcal{C}$ denote the set of CVE identifiers in the current KEV catalog. A node v is *exploited-vulnerable* if $\mathrm{vuln}(v) \cap K \neq \emptyset$.

Exploited reachable set. Define:

$$\mathrm{ER}(G, S, K) = \{v \in \mathrm{Reach}(G, S) \mid \mathrm{vuln}(v) \cap K \neq \emptyset\}. \quad (1)$$

Exploited reachable paths. An *exploited reachable path* is any directed path $\pi = (s, v_1, \ldots, v_k)$ in G with $s \in S$ and $v_k \in \mathrm{ER}(G, S, K)$. Let $\Pi(G, S, K)$ denote the set of all such paths.

V. Formal Problem Formulation

A. Exploited-Exposure Metric

We define the *exploited exposure* of a project as the cardinality of its exploited reachable set, weighted by the number of distinct KEV CVEs:

$$\mathrm{ExExp}(G, S, K) = \sum_{v \in \mathrm{ER}(G,S,K)} |\mathrm{vuln}(v) \cap K|. \quad (2)$$

This metric is zero if and only if no KEV-listed CVE is reachable.

B. Remediation Actions

A *remediation action* $a = (\mathrm{pkg}, \mathrm{ver}_{old}, \mathrm{ver}_{new})$ specifies upgrading package pkg from version ver_{old} to ver_{new}, where ver_{new} is not affected by any CVE in K (per OSV). Applying a to G produces $G' = \mathrm{Apply}(G, a)$, where nodes matching $(\mathrm{pkg}, \mathrm{ver}_{old})$ are replaced by $(\mathrm{pkg}, \mathrm{ver}_{new})$ and edges updated to reflect the new version's resolved dependencies.

Each action has cost $c(a) \in \mathbb{R}_{>0}$; in the simplest model $c(a) = 1$.

C. Risk-Time Objective

Let $K(t)$ denote the KEV catalog at time t (monotone growth), and suppose the defender applies an ordered sequence of actions $\sigma = (a_1, \ldots, a_m)$ at times $t_1 < \cdots < t_m$. Let $G_0 = G$ and $G_i = \mathrm{Apply}(G_{i-1}, a_i)$.

We define a discrete risk-time integral:

$$\mathrm{RT}_{disc}(\sigma, G, S, K) = \sum_{i=0}^{m} \mathrm{ExExp}(G_i, S, K(t_i)) \cdot \Delta t_i, \quad (3)$$

where $\Delta t_i = t_{i+1} - t_i$ with $t_{m+1} = T$.

Optimization problem. Given G, S, K, candidate actions $\mathcal{A}$, and budget B:

$$\min_{\sigma \in \mathrm{Seq}(\mathcal{A}),\, \sum c(a_i) \leq B} \mathrm{RT}_{disc}(\sigma, G, S, K). \quad (4)$$

This is a temporal variant of weighted set cover; we provide a greedy approximation (Section VII).

D. Risk-Reduction Threshold

Define T_ϵ as the earliest time at which exploited exposure drops below threshold ϵ:

$$T_\epsilon = \min\{t \mid \mathrm{ExExp}(G(t), S, K(t)) \leq \epsilon\}. \quad (5)$$

The case $\epsilon = 0$ corresponds to full elimination of exploited reachable exposure.

VI. The KEVGraph Method

A. Graph Construction

Step 1 (lockfile parsing). Given `package-lock.json` (lockfile version 2 or 3), parse the `packages` field to extract resolved package-version pairs and dependency relations. Each entry yields a node $v = (\mathrm{pkg}, \mathrm{ver})$, and each dependency key yields a directed edge.

Step 2 (supplementary resolution via deps.dev). For cases where the lockfile does not fully specify transitive dependencies (e.g., optional or peer dependencies resolved at install time), query the deps.dev API [6] to obtain resolved dependency data. This step is conservative: it adds edges only where lockfile semantics are ambiguous.

B. Vulnerability Join

For each node $v = (\mathrm{pkg}, \mathrm{ver})$, query the OSV API [5] and record $\mathrm{vuln}(v)$. Batch queries and cache responses by (package, version).

C. KEV Labeling

Download the current KEV feed from CISA [2] and extract K. Mark nodes with $\mathrm{vuln}(v) \cap K \neq \emptyset$ as exploited-vulnerable.

D. Candidate Action Generation

For each exploited-vulnerable node v, query the npm registry (or deps.dev) for versions of $\mathrm{pkg}(v)$ that:

1) are not affected by any CVE in $\mathrm{vuln}(v) \cap K$ (verified via OSV), and
2) satisfy parent semver constraints (or, optionally, the nearest semver-compatible version if constraint relaxation is allowed).

Each yields an action $a = (\mathrm{pkg}, \mathrm{ver}_{old}, \mathrm{ver}_{new})$. If no compatible non-vulnerable version exists, record an *unresolvable* exposure.

E. Scoring for Risk-Time Reduction

For each candidate action a, compute marginal exploited exposure reduction:

$$\Delta(a) = \mathrm{ExExp}(G, S, K) - \mathrm{ExExp}(\mathrm{Apply}(G, a), S, K). \quad (6)$$

A simple greedy score is:

$$\mathrm{score}(a) = \frac{\Delta(a)}{c(a)}. \quad (7)$$

In practice we accelerate scoring via maintained reachability sets and path breakage checks.

F. Certificate Extraction

After selecting action a, extract a *remediation certificate* $\mathcal{P}(a)$ containing:

1) action specification $(\text{pkg}, \text{ver}_{old}, \text{ver}_{new})$,
2) broken paths $\Pi_a \subseteq \Pi(G, S, K)$,
3) minimal proof subgraph G_a (union of edges on paths in Π_a, minimized by removing edges not on any path),
4) vulnerability evidence (OSV record + KEV membership for terminal nodes),
5) fix evidence (OSV query confirming ver_{new} is not affected by relevant KEV CVEs).

Minimality. G_a is minimal if removing any edge would cause at least one path in Π_a to become unrepresentable.

G. Certificate Verification

Given certificate $\mathcal{P}(a)$, verification checks:

1) edges in G_a appear in the lockfile-derived graph,
2) each path in Π_a is valid and terminates at an exploited-vulnerable node,
3) OSV confirms the CVE affects the terminal node and KEV contains the CVE,
4) OSV confirms ver_{new} is not affected by the relevant KEV CVEs,
5) applying the version replacement breaks the certified paths.

Verification is deterministic given the same public snapshots.

VII. Algorithms

A. Main Planner

```
Algorithm 1: KEVGraph-Plan(G, S, K, B)
Input: G=(V,E) resolved dependency graph
       S entrypoint set
       K KEV set (CVE IDs)
       B change budget
Output: sigma = sequence of (action,
    certificate)

sigma <- []
budget_remaining <- B
G_curr <- G
while budget_remaining > 0:
    ER <- ComputeExploitedReachable(G_curr,
        S, K)
    if ER is empty: break
    A <- GenerateCandidateActions(G_curr, ER
        , K)
    if A is empty: break
    for each a in A:
        delta[a] <- ExExp(G_curr,S,K) -
            ExExp(Apply(G_curr,a),S,K)
        score[a] <- delta[a] / c(a)
    a_star <- argmax score[a] s.t. c(a) <=
        budget_remaining
    if delta[a_star] == 0: break
    cert <- ExtractCertificate(G_curr, S, K,
         a_star)
    append (a_star, cert) to sigma
    G_curr <- Apply(G_curr, a_star)
    budget_remaining <- budget_remaining - c
        (a_star)
return sigma
```

Complexity. Each iteration computes exploited reachability via BFS ($O(|V| + |E|)$), generates candidates, and scores actions. Iterations are bounded by $\min(B, |\text{ER}_0|)$.

B. Certificate Extraction

```
Algorithm 2: ExtractCertificate(G, S, K, a)
Input:  G, S, K
        a=(pkg, ver_old, ver_new)
Output: certificate P(a)

T_a <- {v in ER(G,S,K) | pkg(v)=pkg and ver(
    v)=ver_old}
Pi_a <- empty set
for each v in T_a:
    paths <- BoundedPathsFromEntrypoints(G,
        S, v, W_max)
    Pi_a <- Pi_a union paths
G_a <- UnionOfPaths(Pi_a)
G_a <- RemoveEdgesNotOnAnyPath(G_a, Pi_a)
vuln_evidence <- {(v, vuln(v) intersect K)
    for v in T_a}
fix_evidence <- OSV_Query(pkg, ver_new)
return Certificate(a, Pi_a, G_a,
    vuln_evidence, fix_evidence)
```

Path enumeration bound. Since path counts may be exponential, certificates use bounded/sampled path sets (W_{max} per terminal node). Soundness holds for the paths included.

C. Verification

```
Algorithm 3: VerifyCertificate(P, lockfile,
    K_snapshot, osv_api)
Input:  P=(a, Pi_a, G_a, vuln_evidence,
    fix_evidence)
Output: VALID or INVALID

G_full <- ParseLockfile(lockfile)
for each edge (u,v) in G_a:
    if (u,v) not in G_full: return INVALID("
        edge not in lockfile graph")
for each path pi in Pi_a:
    if not IsValidPath(G_a, pi): return
        INVALID("invalid path")
    if pi[0] not in S: return INVALID("not
        from entrypoint")
    if pi[-1] not in ER(G_a,S,K_snapshot):
        return INVALID("terminal not
        exploited")
for each (v, cves) in vuln_evidence:
    osv_result <- osv_api.query(pkg(v), ver(
        v))
    for each cve in cves:
        if cve not in osv_result: return
            INVALID("CVE not in OSV record")
        if cve not in K_snapshot: return
            INVALID("CVE not in KEV")
(pkg, ver_old, ver_new) <- a
osv_new <- osv_api.query(pkg, ver_new)
```

```
for each (v, cves) in vuln_evidence:
    for each cve in cves:
        if cve in osv_new: return INVALID("
            fix version still vulnerable")
G_fixed <- ReplaceVersion(G_a, pkg, ver_old,
     ver_new)
for each path pi in Pi_a:
    if PathStillExists(G_fixed, pi): return
        INVALID("path survives after fix")
return VALID
```

D. Scaling Strategies

Bounded BFS. For very large graphs, restrict reachability computation to depth D from entrypoints. Depth bounding trades completeness for speed.

Incremental updates on KEV deltas. When KEV adds new CVEs, update exploited reachability by rechecking only nodes affected by those CVEs.

Path sampling. Bound certificate size by sampling up to W_{max} paths per terminal node and recording this bound in certificate metadata.

VIII. Theoretical Properties

A. Theorem 1: Certificate Soundness

Statement. If $\text{VerifyCertificate}(\mathcal{P}, \text{lockfile}, K, \text{osv})$ returns VALID, then applying action a to the resolved dependency graph G parsed from the lockfile breaks at least one exploited reachable path in $\Pi(G, S, K)$.

Proof sketch. Valid verification implies: (i) G_a is a subgraph of G; (ii) each certified path $\pi \in \Pi_a$ is a valid path from an entrypoint to a KEV-labeled vulnerable terminal node; (iii) OSV and KEV checks confirm the vulnerability evidence; (iv) the fix version is not vulnerable to those KEV CVEs; and (v) the breakage check confirms that the certified paths do not survive under the version replacement in G_a. With $|\Pi_a| \geq 1$, at least one exploited reachable path in G is broken by a in the model. □

B. Theorem 2: Risk Monotonicity (Conditional)

Statement. Under the model, applying any valid action a does not increase exploited exposure:

$$\text{ExExp}(\text{Apply}(G, a), S, K) \leq \text{ExExp}(G, S, K), \quad (8)$$

provided (a) ver_{new} does not include the relevant KEV CVEs and (b) newly introduced transitive dependencies do not introduce new exploited-vulnerable reachable nodes.

Proof sketch. The action removes the KEV CVEs associated with the upgraded nodes. The inequality follows if no new exploited-vulnerable reachable nodes are introduced by the upgrade's transitive closure (precondition checked via OSV over that closure). □

C. Informal Greedy Approximation Argument

The risk-time minimization resembles weighted set cover where actions "cover" exploited paths. Greedy benefit-per-cost achieves a $\ln(n) + 1$ approximation for static set cover [16]. Our objective is temporal and graph-changing; we adaptively recompute marginal benefits. While we do not prove a tight bound for the temporal integral, the adaptive greedy structure aligns with well-studied results for submodular maximization under constraints [17]. Establishing submodularity for exploited-exposure reduction under realistic dependency update semantics is left for future work.

IX. Evaluation Plan

We describe a reproducible evaluation using only public data.

A. Dataset Selection

Select 200–500 public GitHub repositories meeting:

- Primary language JavaScript/TypeScript,
- contains `package-lock.json` (v2 or v3) at repo root,
- at least 100 stars,
- at least 50 direct dependencies,
- last commit within 12 months.

Use the GitHub API to search, and sample from a bounded popularity window to reduce bias. Record commit SHAs.

B. Dependency Graph Extraction

For each repository: clone at the recorded SHA, parse `package-lock.json` to extract G, and record graph statistics ($|V|, |E|$, max depth).

C. Vulnerability Mapping

Query OSV for each node (batch + cache), record CVEs per node, and intersect with KEV to identify exploited-vulnerable nodes.

D. KEV Timeline Reconstruction

Using KEV metadata fields (e.g., date added), reconstruct $K(t)$ snapshots monthly to simulate defender knowledge over time.

E. Baselines

Compare against:

- **Severity-first:** prioritize by CVSS base score (where available).
- **Direct-deps-first:** fix direct deps before transitives.
- **Newest-first:** apply most recent available fix first.
- **Centrality-first:** prioritize by graph centrality.

All baselines use the same candidate action set and cost model.

F. Metrics

1) Time-to-ϵ exploited risk (T_ϵ) for $\epsilon \in \{0, 1, 5\}$,
2) number of actions required to reach ϵ,
3) risk-time integral RT_{disc},
4) verification time per certificate,
5) certificate size (nodes/edges),
6) robustness under dependency noise (edge perturbations).

G. Ablations

- Remove KEV signal (treat all CVEs equally),
- remove certificate minimization,
- compare risk-time objective vs. static coverage objective.

H. Reproducibility Artifacts

Publish: dataset manifest (repo + SHA), lockfile hashes, KEV snapshots, cached OSV results, scripts, Dockerfile, and raw CSV outputs.

X. Discussion

A. Auditability and Incident Response

Remediation certificates are designed as audit artifacts. A downstream consumer or regulator can verify that a vendor addressed specific exploited vulnerabilities without access to internal systems. Certificates reference public data (KEV CVEs, OSV advisories, dependency edges) and can be verified using public APIs.

B. Cross-Organizational Trust

In regulated sectors and federal contexts, remediation timelines for exploited vulnerabilities are operationally enforced [4]. Proof-carrying remediation enables vendors to demonstrate compliance to downstream consumers without disclosing proprietary implementation details.

C. Alignment with Federal Guidance

EO 14028 emphasizes software supply chain security and timely remediation [3]. NIST SSDF requires responding to vulnerabilities in components used [12]. KEVGraph operationalizes these requirements by producing optimized, verifiable remediation plans grounded in exploitation evidence.

XI. Limitations

1) **Package-level reachability only.** We do not model call-graph reachability; exposure may be overestimated.
2) **Lockfile fidelity.** Lockfiles may diverge from installed graphs due to optional/platform resolution.
3) **OSV completeness.** Missing/lagging advisories can cause false negatives.
4) **Upgrade feasibility.** Compatibility and runtime breakage are not guaranteed by vulnerability absence.
5) **npm-only.** Extending to other ecosystems requires parser/registry changes.
6) **No formal approximation bound.** We provide an informal greedy rationale only.

XII. Ethical Considerations

This work is strictly defensive. It uses public vulnerability metadata and dependency graphs, and does not produce exploit code or operational exploitation guidance. Evaluation uses public repositories and public APIs.

XIII. Patentability and Intellectual Property Discussion

This section identifies method components that *may* be patentable relative to cited public prior art; it is not a legal opinion. A professional prior art search is required.

A. Potentially Patentable Components

1) **Proof-carrying remediation certificate construction:** extracting minimal proof subgraphs showing exploited-path breakage by upgrade actions.
2) **Certificate verification protocol:** deterministic third-party verification using only public snapshots.
3) **Risk-time optimization under KEV deltas:** temporal objective conditioned on evolving exploited-vulnerability catalogs.
4) **Incremental exploited-reachability maintenance:** updates driven by catalog deltas rather than full recomputation.

B. Proposed Claim Themes (Hypotheses)

1) Methods for generating minimal proof subgraphs certifying elimination of exploited reachable paths by upgrade actions.
2) Methods for verifying remediation certificates against public vulnerability/exploitation catalogs without internal system access.
3) Systems optimizing remediation sequences to minimize risk-time integrals over exploited reachable exposure.
4) Incremental maintenance of exploited reachability under exploitation-catalog updates.
5) Certificate data structures encoding actions, proof subgraphs, and public evidence pointers.
6) Scoring candidate actions by marginal exploited-path breakage normalized by cost.
7) Machine-readable remediation certificates complementary to SBOMs, verifiable by downstream consumers.
8) Historical exploitation-timeline reconstruction for simulation/evaluation.
9) Bounded certificate extraction via path sampling with explicit soundness conditions.
10) Integrated systems combining dependency resolution, exploited-catalog integration, and proof artifact generation.

Appendix: Self-Audit

Originality Claims (Specific Points)

1) Exploited-exposure metric grounded in KEV membership and dependency reachability.
2) Risk-time integral objective conditioned on a time-evolving exploited-catalog signal.
3) Proof-carrying remediation certificates with explicit verification.
4) Minimality definition for proof subgraphs and extraction procedure.
5) Integrated framework linking public dependency graphs, OSV mapping, KEV exploitation, optimization, and verification.

Plagiarism Risk Assessment (How Avoided)

1) Standard graph reachability and dependency modeling: common notation; cites empirical dependency studies.
2) Greedy set cover and submodular references: cites original theory sources.
3) Public data source descriptions (KEV/OSV/deps.dev/npm): cites official documentation.
4) Comparisons to reachability work (e.g., Eclipse Steady): cites published paper; does not reuse proprietary method detail.
5) SBOM/SLSA comparisons: cites standards; avoids reproducing specification text.

Confirmations

- Public data only: lockfiles from public repos; OSV, deps.dev, KEV, npm registry.
- No employer information or proprietary tooling.
- No fabricated experimental results (evaluation plan only).

Citation-Needed Placeholders

None in this draft; URLs should be verified at access time.

Acknowledgments

(Add acknowledgments if any; otherwise remove.)

References

[1] MITRE Corporation, "CVE-2021-44228," Common Vulnerabilities and Exposures (CVE). [Online]. Available: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-44228

[2] Cybersecurity and Infrastructure Security Agency (CISA), "Known Exploited Vulnerabilities Catalog." [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[3] Executive Office of the President, "Executive Order 14028: Improving the Nation's Cybersecurity," May 12, 2021. [Online]. Available: https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/

[4] CISA, "Binding Operational Directive 22-01: Reducing the Significant Risk of Known Exploited Vulnerabilities," Nov. 3, 2021. [Online]. Available: https://www.cisa.gov/binding-operational-directive-22-01

[5] Open Source Vulnerabilities (OSV), "OSV," [Online]. Available: https://osv.dev

[6] Google, "deps.dev: Open Source Insights," [Online]. Available: https://deps.dev

[7] npm Documentation, "`package-lock.json`," [Online]. Available: https://docs.npmjs.com/cli/v10/configuring-npm/package-lock-json

[8] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid, and M. Roytman, "Improving vulnerability remediation through better exploit prediction," *Journal of Cybersecurity*, vol. 6, no. 1, 2020.

[9] J. Spring, E. Hatleback, A. Householder, A. Manion, and D. Shick, "Prioritizing vulnerability response: A stakeholder-specific vulnerability categorization (SSVC)," Software Engineering Institute, Carnegie Mellon University, 2020.

[10] H. Plate, S. E. Ponta, and A. Sabetta, "Impact assessment for vulnerabilities in open-source software libraries," in *Proc. IEEE International Conf. on Software Maintenance and Evolution (ICSME)*, 2015, pp. 411–420.

[11] Supply-chain Levels for Software Artifacts (SLSA), "SLSA," [Online]. Available: https://slsa.dev

[12] NIST, "SP 800-218: Secure Software Development Framework (SSDF) Version 1.1," Feb. 2022.

[13] NIST, "Cybersecurity Framework," [Online]. Available: https://www.nist.gov/cyberframework

[14] OWASP Foundation, "CycloneDX," [Online]. Available: https://cyclonedx.org

[15] Linux Foundation, "SPDX," [Online]. Available: https://spdx.dev

[16] V. Chvátal, "A greedy heuristic for the set-covering problem," *Mathematics of Operations Research*, vol. 4, no. 3, pp. 233–235, 1979.

[17] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[18] R. Kikas, G. Gousios, M. Dumas, and D. Pfahl, "Structure and evolution of package dependency networks," in *Proc. IEEE/ACM MSR*, 2017, pp. 102–112.

[19] A. Decan, T. Mens, and E. Constantinou, "On the impact of security vulnerabilities in the npm package dependency network," in *Proc. IEEE/ACM MSR*, 2018, pp. 181–191.

[20] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *USENIX Security Symposium*, 2019.

[21] NIST, "National Vulnerability Database (NVD)," [Online]. Available: https://nvd.nist.gov

[22] OpenSSF, "Scorecard: Security health metrics for open source," [Online]. Available: https://securityscorecards.dev

[23] J. Cox, E. Bouwers, M. van Eekelen, and J. Visser, "Measuring dependency freshness in software systems," in *Proc. IEEE/ACM ICSE (Companion)*, 2015, pp. 109–118.

[24] CISA, "Secure by Design," [Online]. Available: https://www.cisa.gov/securebydesign